

ICS2210 Course Project

AVL Trees vs Red-Black Trees vs Skip Lists

Dave Galea 0283404L

May 2026

Contents

Statement of Completion and Plagiarism Declaration	3
1 Introduction	5
2 Array Creation & Instantiation	6
2.1 First Array	6
2.2 Second Array	6
3 Knuth Shuffle	7
4 AVL Tree	8
4.1 Utility Functions	8
4.2 Insertion	10
4.3 Rotations	12
5 Red-Black Tree	14
5.1 Utility Functions	14
5.2 Insertion	15
5.3 Red Red Conflict Repair	17
5.4 Rotation	18
6 Skip List	20
6.1 Promotion by Coin Flips	20
6.2 Search and Insertion	21
7 Statistics and Evaluation	24
7.1 Collected Measures	24
7.2 Step Statistics	24
7.3 Rotation and Promotion Statistics	25
7.4 Tree and List Information	25
7.5 A Brief Comparison	25

7.6 Real-World Preference	26
A Generative AI Usage Appendix	27

Statement of Completion and Plagiarism Declaration

Statement of Completion

Item	Completed (Yes/No/Partial)
Created first array of integers	Yes
Knuth shuffle	Yes
Insertion in AVL tree	Yes
AVL tree insertion statistics	Yes
Insertion in Red-Black tree	Yes
Red-Black tree insertion statistics	Yes
Insertion in Skip List	Yes
Skip List insertion statistics	Yes
Discussion comparing data structures	Yes

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

Declaration

Plagiarism is defined as “the unacknowledged use, as one’s own work, of work of another person, whether or not such work has been published” (Regulations Governing Conduct at Examinations, 1997, Regulation 1 (viii), University of Malta).

I / ~~We~~*, the undersigned, declare that the [assignment / Assigned Practical Task report / Final Year Project report] submitted is my / ~~our~~* work, except where acknowledged and referenced.

I / ~~We~~* understand that the penalties for making a false declaration may include, but are not limited to, loss of marks; cancellation of examination results; enforced suspension of studies; or expulsion from the degree programme.

Work submitted without this signed declaration will not be corrected, and will be given zero marks.

* Delete as appropriate.

(N.B. If the assignment is meant to be submitted anonymously, please sign this form and submit it to the Departmental Officer separately from the assignment).

Dave Grech

Student Name

[Signature]

Signature

Student Name

Signature

Student Name

Signature

Student Name

Signature

ICS2210

Course Code

Course Project

Title of work submitted

29th May 2026

Date

Chapter 1

Introduction

This project compares three data structures: an AVL tree, a Red-Black tree, and a Skip List. The practical task is to first create an array containing the integers from 1 to 10,000, randomise it using an own implementation of the Knuth shuffle, and insert those values into all three data structures. A second array of 2,000 random integers in the range $[0, 20000]$ is then generated and inserted into the already populated structures. Statistics are collected for the second insertion phase only.

The implementation is written in C++. The code is organised into separate header and source files for each major component:

- `KnuthShuffle.h/.cpp`: Knuth shuffle implementation.
- `AVL_Tree.h/.cpp`: AVL tree implementation and insertion statistics.
- `RB_Tree.h/.cpp`: Red-Black tree implementation and insertion statistics.
- `SkipList.h/.cpp`: Skip List implementation and insertion statistics.
- `Statistics.h/.cpp`: shared statistical calculations and formatted output.
- `main.cpp`: program driver that creates arrays, inserts values, stores statistics, and prints the final analysis.

Chapter 2

Array Creation & Instantiation

2.1 First Array

The first array contains exactly 10,000 integers, and has no duplicates.

```
std::vector<int> first_arr(10000);

for (int idx = 0; idx < 10000; idx++) {
    int idx_val = idx + 1;
    first_arr[idx] = idx_val;
}
```

This array is shuffled before any insertions are made. The shuffled order prevents the initial build phase from being a sorted insertion sequence.

2.2 Second Array

The second array contains 2,000 random integers in the required range [0,20000]. This array may contain duplicates, as well as values already existing in the first array.

```
std::vector<int> second_arr(2000);

for (int idx = 0; idx < 2000; idx++) {
    int idx_val = rand() % 20001;
    second_arr[idx] = idx_val;
}
```

This array will be used for insertion into the 3 implemented data structures, and relevant statistics when doing so will be monitored, as discussed in later sections.

Chapter 3

Knuth Shuffle

The Knuth shuffle is implemented by moving backwards from the end of the array towards the beginning. At each index, the algorithm selects a random earlier index, including the current index, and swaps the two elements.

```
void Knuth_Shuffle(std::vector<int>& arr) {  
  
    // position shuffle_line = current dividing line  
    // each iteration, position shuffle_line decreases by 1  
    // item toshuffle is random, swaps place with position i  
  
    // Based on video explaining Knuth Shuffle by James Cutajar  
  
    int last_index = arr.size() - 1; //idx 9999 = size of 10000 - 1  
    int first_index = 0;  
  
    srand(1) ; // seed for rand()  
  
    for (int i = last_index; i > 0; i--) {  
        int shuffle_line = i;  
        int to_shuffle = rand() % (shuffle_line - first_index + 1) +  
            first_index;  
        std::swap(arr[shuffle_line], arr[to_shuffle]);  
    }  
}
```

The variable `shuffle_line` represents the current boundary between the already shuffled portion and the remaining unshuffled portion. The randomly chosen index `to_shuffle` is selected from the range $[0, i]$, and the two positions are swapped. The idea for this implementation of the Knuth Shuffle was inspired from a very easy to understand [video by James Cutajar](#).

Chapter 4

AVL Tree

The AVL tree is built from nodes that store an integer value, the height of the subtree rooted at that node, and two child pointers. The left pointer refers to values smaller than the current node, while the right pointer refers to values that are greater than or equal to the current node.

```
struct AVLNode {
    int data;
    int height;

    AVLNode* left;
    AVLNode* right;
};
```

The tree itself stores a pointer to the root node. When the tree is first created, the root is set to `nullptr`, meaning that the tree is initially empty.

```
AVLTree::AVLTree() {
    root = nullptr;
}
```

4.1 Utility Functions

The AVL tree implementation uses several helper functions:

- `create_node`: Creates and initialises a new node.
- `get_node_height`: Returns a node's height.
- `get_tree_height`: Returns the full tree height.
- `count_leaves`: Counts leaf nodes recursively.
- `get_leaves`: Starts leaf counting from root.
- `get_max`: Returns the larger value.
- `get_balance_factor`: Calculates left-right height difference.

```

AVLNode* AVLTree::create_node(int value) {

    AVLNode* new_node = new AVLNode;

    new_node->data = value;
    new_node->height = 1;
    new_node->left = nullptr;
    new_node->right = nullptr;

    return new_node;
}

// get height of specific node
int AVLTree::get_node_height(AVLNode* node) {

    if (node == nullptr) {
        return 0;
    }

    return node->height;
}

// find max tree hight with root
int AVLTree::get_tree_height() {
    return get_node_height(root);
}

// count leaves
int AVLTree::count_leaves(AVLNode* node) {

    if (node == nullptr) {
        return 0;
    }

    if (node->left == nullptr && node->right == nullptr) {
        return 1;
    }

    return count_leaves(node->left) + count_leaves(node->right);
}

// find total leaves by counting from root
int AVLTree::get_leaves() {
    return count_leaves(root);
}

// maximum
int AVLTree::get_max(int val_one, int val_two) {

    if (val_one > val_two) {
        return val_one;
    }
    else {
        return val_two;
    }
}

```

```

// balance factor = left - right height
int AVLTree::get_balance_factor(AVLNode* node) {

    if (node == nullptr) {
        return 0;
    }

    return get_node_height(node->left) - get_node_height(node->
        right);
}

```

4.2 Insertion

The AVL insertion routine first behaves like a standard binary search tree insertion. If the inserted value is smaller than the current node, insertion continues on the left. Otherwise, insertion continues on the right. Equal values are therefore inserted in the right subtree.

Each visited node increments the step counter. After the recursive insertion returns, the node height is recalculated, the balance factor is checked, and rotations are applied where necessary.

```

// insert function
InsertResult AVLTree::insert(int value) {

    InsertResult result;

    result.steps = 0;
    result.rotations = 0;
    result.promotions = 0;

    root = insert_node(root, value, result);

    return result;
}

```

```

AVLNode* AVLTree::insert_node(AVLNode* node, int value,
    InsertResult& result) {

    // if empty position is found, create the new node
    if (node == nullptr) {
        AVLNode* new_node = create_node(value);
        return new_node;
    }

    //node visited = step added
    result.steps++;

    // Case 1: Value is smaller
    if (value < node->data) {
        node->left = insert_node(node->left, value, result);
    }

    // Case 2: Value is equal or larger
    else {
        node->right = insert_node(node->right, value, result);
    }
}

```

```

// update height after insertion
int left_height = get_node_height(node->left);
int right_height = get_node_height(node->right);
node->height = 1 + get_max(left_height, right_height);

// balance factor = height(left subtree) - height(right subtree)
int balance_factor = get_balance_factor(node);

// if balance factor is more than 1 or less than -1, AVL
// condition violated
// =====
// LL CASE :value in left subtree of left child

if (balance_factor > 1 && value < node->left->data) {

    if (result.rotations == 0) {
        result.rotations++;
    }
    return rotate_right(node);
}
// =====
// RR case: value in right subtree of right child

if (balance_factor < -1 && value >= node->right->data) {

    if (result.rotations == 0) {
        result.rotations++;
    }
    return rotate_left(node);
}
// =====
// LR case: value in right subtree of left child

if (balance_factor > 1 && value >= node->left->data) {

    if (result.rotations == 0) {
        result.rotations++;
    }
    node->left = rotate_left(node->left);
    return rotate_right(node);
}
// =====
// RL case: value in left subtree of right child

if (balance_factor < -1 && value < node->right->data) {

    if (result.rotations == 0) {
        result.rotations++;
    }
    node->right = rotate_right(node->right);
    return rotate_left(node);
}
// =====
// Case already balanced: return node ptr

return node;
}

```

The four standard AVL cases are handled: left-left, right-right, left-right, and right-left. A double rotation (LR or RL) is counted as one rotation in the statistics.

4.3 Rotations

A left rotation is used when a subtree has become too heavy on the right. The right child is promoted to become the new root of the local subtree. The original node becomes the left child of that promoted node. The promoted node's original left subtree is moved to become the right subtree of the original node.

```
AVLNode* AVLTree::rotate_left(AVLNode* node) {

    AVLNode* right_child = node->right;
    AVLNode* middle_subtree = right_child->left;

    right_child->left = node;
    node->right = middle_subtree;

    // update heights after rotation
    int node_left_height = get_node_height(node->left);
    int node_right_height = get_node_height(node->right);

    node->height = 1 + get_max(node_left_height, node_right_height)
        ;

    int right_left_height = get_node_height(right_child->left);
    int right_right_height = get_node_height(right_child->right);

    right_child->height = 1 + get_max(right_left_height,
        right_right_height);

    return right_child;
}
```

A right rotation is the opposite operation. It is used when a subtree has become too heavy on the left. The left child is promoted, the original node becomes the right child of the promoted node, and the promoted node's original right subtree becomes the left subtree of the original node.

```
AVLNode* AVLTree::rotate_right(AVLNode* node) {

    AVLNode* left_child = node->left;
    AVLNode* middle_subtree = left_child->right;

    left_child->right = node;
    node->left = middle_subtree;

    // update heights
    int node_left_height = get_node_height(node->left);
    int node_right_height = get_node_height(node->right);

    node->height = 1 + get_max(node_left_height, node_right_height)
        ;

    int left_left_height = get_node_height(left_child->left);
    int left_right_height = get_node_height(left_child->right);
}
```

```
        left_child->height = 1 + get_max(left_left_height,
            left_right_height);

    return left_child;
}
```

Both rotation functions return the new root of the local subtree. This returned pointer is then attached back to the parent node through the recursive insertion function. If the rotated node was the root of the whole tree, the returned pointer becomes the new tree root.

Chapter 5

Red-Black Tree

The Red-Black tree node stores its value, colour, child pointers, and parent pointer. The parent pointer is needed because rotations in a Red-Black tree must reconnect the rotated subtree to the old parent of the rotated node.

```
struct RBNode {
    int data;
    bool is_red;

    RBNode* left;
    RBNode* right;
    RBNode* parent;
};
```

In the same fashion as in the AVL Tree, the tree itself stores a pointer to the root node, and when it is first created, the root is set to nullptr. This initializes an empty tree

```
RedBlackTree::RedBlackTree() {
    root = nullptr;
}
```

5.1 Utility Functions

Most of the utility functions used in the AVL tree are reused in the Red-Black tree, such as the functions for height calculation, leaf counting, and finding the maximum of two values. The main difference is that the Red-Black tree does not need a balance factor function, since balancing is based on node colours instead.

The Red-Black specific functions are:

- `create_node`: Creates a new red node.
- `node_is_red`: Checks node colour.
- `flip_colours`: Splits two-red-child nodes.

```
// creates a new Red-Black node
// new nodes are inserted as red first
RBNode* RedBlackTree::create_node(int value) {
```

```

    RBNode* new_node = new RBNode;

    new_node->data = value;
    new_node->is_red = true;

    new_node->left = nullptr;
    new_node->right = nullptr;
    new_node->parent = nullptr;

    return new_node;
}

// treat null nodes as black
bool RedBlackTree::node_is_red(RBNode* node) {

    if (node == nullptr) {
        return false;
    }

    return node->is_red;
}

// flip colours for top-down insertion
// current node becomes red
// both red children become black
void RedBlackTree::flip_colours(RBNode* node) {

    node->is_red = true;

    if (node->left != nullptr) {
        node->left->is_red = false;
    }

    if (node->right != nullptr) {
        node->right->is_red = false;
    }
}

```

5.2 Insertion

The implementation uses a top-down Red-Black insertion approach. While searching for the insertion position, the algorithm checks whether the current node has two red children. If it does, the colours are flipped before the search continues.

```

// Top down approach taken: fix as you go along
InsertResult RedBlackTree::insert(int value) {

    InsertResult result;
    result.steps = 0;
    result.rotations = 0;
    result.promotions = 0;
    RBNode* parent_node = nullptr;
    RBNode* current_node = root;

    // If tree empty -> insert root as black

```

```

if (root == nullptr) {
    root = create_node(value);
    root->is_red = false;

    return result;
}

// search down tree like a normal BST
// but on the way down, split nodes that have two red children
while (current_node != nullptr) {

    // node visited = step added
    result.steps++;

    // top-down colour flip
    // current node two red children -> current node = red, both
    // children = black
    if (node_is_red(current_node->left) && node_is_red(current_node->right)) {

        flip_colours(current_node);

        // root must always stay black
        if (current_node == root) {
            current_node->is_red = false;
        }

        // if colour flip created red parent and red child,
        // fix it immediately before going further down
        else if (node_is_red(current_node->parent)) {
            fix_red_red(current_node, result);
        }
    }

    parent_node = current_node;

    // normal BST search: smaller values go left, equal or larger
    // right
    if (value < current_node->data) {
        current_node = current_node->left;
    } else {
        current_node = current_node->right;
    }
}

// IF current_node is null, insertion point found
RBNode* new_node = create_node(value);
new_node->parent = parent_node;

if (value < parent_node->data) {
    parent_node->left = new_node;
}
else {
    parent_node->right = new_node;
}

// new node is red
// if parent is red, we have a red-red violation

```

```

    // this can be fixed with one single or double rotation

    if (node_is_red(parent_node)) {
        fix_red_red(new_node, result);
    }

    // root must always be black
    root->is_red = false;

    return result;
}

```

5.3 Red Red Conflict Repair

The `fix_red_red` function distinguishes whether the parent is the left or right child of the grandparent, and whether the new node forms an outside or inside case. Outside cases require one rotation, while inside cases require a double rotation. In either case, the insertion records one balancing event for that fix.

```

// fixes red-red violation: node and parent both red
void RedBlackTree::fix_red_red(RBNode* node, InsertResult& result) {

    RBNode* parent_node = node->parent;

    if (parent_node == nullptr) {
        return;
    }

    RBNode* grandparent_node = parent_node->parent;

    if (grandparent_node == nullptr) {
        parent_node->is_red = false;
        return;
    }

    // parent is left child of grandparent=====
    if (parent_node == grandparent_node->left) {

        // LL CASE =====
        // node is outside grandparent
        if (node == parent_node->left) {

            parent_node->is_red = false;
            grandparent_node->is_red = true;

            rotate_right(grandparent_node);

            result.rotations++;
        }

        // LR case =====
        // node is inside grandparent

        else {

            node->is_red = false;

```

```

        grandparent_node->is_red = true;

        rotate_left(parent_node);
        rotate_right(grandparent_node);

        result.rotations++;
    }
}

// parent is right child of grandparent
// =====
else {

    // RR case =====
    // node is outside grandparent

    if (node == parent_node->right) {

        parent_node->is_red = false;
        grandparent_node->is_red = true;

        rotate_left(grandparent_node);

        result.rotations++;
    }

    // RL case =====
    // node is inside grandparent

    else {
        node->is_red = false;
        grandparent_node->is_red = true;

        rotate_right(parent_node);
        rotate_left(grandparent_node);

        result.rotations++;
    }
}
}

```

5.4 Rotation

The Red-Black rotation routines are more involved than the AVL rotations because each affected parent pointer must be updated. The old parent of the rotated node must also be connected to the new subtree root.

```

// LEFT ROTATION
// right child moves up, current node goes left
void RedBlackTree::rotate_left(RBNode* node) {

    RBNode* right_child = node->right;
    RBNode* middle_subtree = right_child->left;

    right_child->left = node;
    node->right = middle_subtree;
}

```

```

// update parent of middle subtree
if (middle_subtree != nullptr) {
    middle_subtree->parent = node;
}

// connect right_child to old parent of node
right_child->parent = node->parent;

//Case 1: node was root: right_child becomes root
if (node->parent == nullptr) {
    root = right_child;
}
// Case 2: node was left child: update parent's left
else if (node == node->parent->left) {
    node->parent->left = right_child;
}
// Case 3: node was right child: update parent's right
else {
    node->parent->right = right_child;
}
// node is now child of right_child
node->parent = right_child;
}

// RIGHT ROTATION
// left child moves up, current node goes down right
void RedBlackTree::rotate_right(RBNode* node) {

    RBNode* left_child = node->left;
    RBNode* middle_subtree = left_child->right;

    left_child->right = node;
    node->left = middle_subtree;

    // update parent of middle subtree
    if (middle_subtree != nullptr) {
        middle_subtree->parent = node;
    }

    // connect left_child to old parent of node
    left_child->parent = node->parent;

    //Case 1: node was root: left_child becomes root
    if (node->parent == nullptr) {
        root = left_child;
    }
    // Case 2: node was left child: update parent's left
    else if (node == node->parent->left) {
        node->parent->left = left_child;
    }
    // Case 3: node was right child: update parent's right
    else {
        node->parent->right = left_child;
    }
    // node is now child of left_child
    node->parent = left_child;
}

```

Chapter 6

Skip List

The Skip List is implemented using nodes that contain a value and a vector of forward pointers.

```
struct SkipNode {
    int data;
    std::vector<SkipNode*> forward;
};
```

The list begins with a sentinel node storing INT_MIN. The maximum level is set to 32 as done so in Skip List Cookbook written by William Pugh , and the current highest used level begins at 1.

```
// start with empty head node
// INT_MIN is used as the -infinity start value
SkipList::SkipList() {

    max_level = 32; // as implemented in the skip list cookbook by
                   William Pugh
    highest_current_level = 1;

    start_node = create_node(INT_MIN, max_level);
}
```

6.1 Promotion by Coin Flips

A new node always appears at the bottom level. Each heads result after performing coin flip promotes it by one additional level, until either tails happens or the maximum level is reached. Every upward promotion is counted in the statistics.

```
// flip a coin
// Heads means promote
// Tails means stop promoting
int SkipList::flip_coin() {

    if (rand() % 2 == 0) {
        return Heads;
    }
    else {
        return Tails;
    }
}
```

```

// generate level for the new node
// level 1 means bottom list only
// each Heads promotes the node one level higher
int SkipList::generate_level(InsertResult& result) {

    int generated_level = 1;

    while (flip_coin() == Heads && generated_level < max_level) {
        generated_level++;
        result.promotions++;
    }

    return generated_level;
}

```

6.2 Search and Insertion

The main insertion function controls the full insertion process. It first prepares the statistics record and the update array. It then searches for the insertion position, generates a random level for the new node, updates the current highest level if needed, and finally links the node into the list.

```

// main insert function
// searches for position, generates level, then inserts node
InsertResult SkipList::insert(int value) {

    InsertResult result;

    result.steps = 0;
    result.rotations = 0;
    result.promotions = 0;

    std::vector<SkipNode*> update(max_level);

    // search for insertion position
    search_insert(value, update, result);

    // randomly generate node level using coin flips
    int generated_level = generate_level(result);

    // if new node has more levels than current skip list,
    // start_node becomes the previous node for those new levels
    // adapted from pseudocode from skiplist cookbook
    if (generated_level > highest_current_level) {

        for (int level = highest_current_level; level < generated_level
            ; level++) {
            update[level] = start_node;
        }

        highest_current_level = generated_level;
    }

    // insert the node into each required level
    create_and_insert(value, generated_level, update);
}

```

```

        return result;
    }

```

The `search_insert` function searches from the highest current level downwards. At each level, it moves forward while the next node is less than or equal to the value being inserted. The `update` array stores the previous node at each level, so the new node can later be linked into the correct places.

```

// searches for the position to insert the value
// update[level] stores the node before the insertion point at each
// level
// structure / pseudo code from skiplist cookbook
SkipNode* SkipList::search_insert(
    int value,
    std::vector<SkipNode*>& update,
    InsertResult& result
) {

    SkipNode* current_node = start_node;

    // start from highest current level
    // move right while the next value is less than or equal to value
    // equal values go right so duplicates can be inserted
    for (int level = highest_current_level - 1; level >= 0; level--) {

        while (
            current_node->forward[level] != nullptr &&
            current_node->forward[level]->data <= value
        ) {
            current_node = current_node->forward[level];

            // each forward movement counts as one step

            result.steps++;
        }

        // store node before insertion point at this level

        update[level] = current_node;
    }

    return current_node;
}

```

After the search is complete, `create_and_insert` creates the new node and updates the forward pointers at each level used by that node.

```

// creates the new node and updates the forward pointers
void SkipList::create_and_insert(
    int value,
    int node_level,
    std::vector<SkipNode*>& update
) {

    SkipNode* new_node = create_node(value, node_level);

    for (int level = 0; level < node_level; level++) {

```

```
new_node->forward[level] = update[level]->forward[level];
update[level]->forward[level] = new_node;
    }
}
```

Your current section already has the right material, but it mixes results, interpretation, and final preference too much. A clearer layout is:

1. show steps, 2. show rotations/promotions, 3. show tree/list information, 4. briefly discuss each table, 5. then give the real-world preference.

The wording below keeps your conclusion: ****AVL is the best middle ground between performance and implementation complexity****, while Red-Black gives only small performance benefits for noticeably more implementation work. The numbers match your current pasted results.

Chapter 7

Statistics and Evaluation

7.1 Collected Measures

The project requires statistics for the second insertion phase, where 2,000 random integers are inserted after the first 10,000 shuffled integers have already been added to each data structure.

For each insertion, the program records:

- the number of steps needed to reach the insertion position;
- the number of rotations for AVL and Red-Black trees;
- the number of promotions for the Skip List.

For each recorded list of values, the minimum, maximum, mean, standard deviation, and median are calculated.

7.2 Step Statistics

The table below shows the step statistics for the 2,000 second-array insertions.

Table 7.1: Step statistics for the 2,000 second-array insertions

Structure	Min	Max	Mean	Std. Dev.	Median
AVL Tree	11	16	14.159	1.009	14
Red-Black Tree	11	18	14.131	1.120	14
Skip List	2	27	11.728	3.649	11

The AVL tree and Red-Black tree had almost identical mean step counts. The Red-Black tree was slightly lower on average, but the difference was very small.

The Skip List had the lowest mean number of steps in this run. However, it also had the highest maximum and standard deviation. This shows that although it performed well on average, its results were less consistent than the two tree structures.

7.3 Rotation and Promotion Statistics

The table below shows the balancing work performed during the second insertion phase. For AVL and Red-Black trees, this means rotations. For the Skip List, this means promotions.

Table 7.2: Rotation and promotion statistics for the 2,000 second-array insertions

Structure / Measure	Min	Max	Mean	Std. Dev.	Median
AVL rotations	0	1	0.4595	0.498	0
Red-Black rotations	0	2	0.3990	0.513	0
Skip List promotions	0	10	0.9980	1.383	1

The Red-Black tree had a lower average number of rotations than the AVL tree. This suggests that Red-Black insertion needed slightly less rebalancing work. However, the difference was small. The AVL tree averaged 0.4595 rotations, while the Red-Black tree averaged 0.3990 rotations.

7.4 Tree and List Information

The table below shows the final height or level count after all insertions were complete.

Table 7.3: Final structure information after all insertions

Structure	Height / Levels	Leaves
AVL Tree	16	5137
Red-Black Tree	17	5138
Skip List	16	Not applicable

The AVL tree had the lowest tree height, ending with height 16. The Red-Black tree ended with height 17. This is consistent with the AVL tree being more strictly balanced.

7.5 A Brief Comparison

The AVL tree gave very consistent results. Its mean step count was almost the same as the Red-Black tree, and it had the lowest final height, but it required slightly more rotations because it keeps the tree more strictly balanced. In terms of implementation, the AVL tree was moderately difficult, since it needs height updates and four rotation cases, but the logic is still easier to follow than the Red-Black tree.

The Red Black tree also performed well. It had a slightly lower mean step count than the AVL tree and fewer rotations on average, which shows that it needed slightly less rebalancing work during insertion. However, it was the most difficult to implement because it required colour handling, red-red conflict repair, parent pointers, and more careful rotation logic. Overall, the AVL Tree and the Red Black Tree differed slightly, and in my opinion, the Red-Black tree starts

to have diminishing returns on performance with respect to difficulty of implementation due to all the different cases; especially if deletion is required in said implementation.

The Skip List had the lowest mean number of steps in this particular run. However, it also had the highest maximum step count and standard deviation, meaning that its results were less consistent. Its implementation was simpler than the tree structures because it does not require rotations, but its behaviour depends on random promotions.

7.6 Real-World Preference

For performance-critical situations:

For performance-critical systems with many insertions and updates, I would prefer the Red-Black tree. Ultimately, in systems where performance and outcome is the goal, the best structure has to be used, even if marginal improvements are made. In the results, it had the lowest mean step count between the two trees and required fewer rotations on average than the AVL tree. Its balancing is also deterministic, unlike the Skip List, so it gives more predictable worst-case behaviour.

For low-performance requirement systems:

For systems with low performance requirements, the Skip List can be a good option when simplicity is more important than strict consistency of outcome. It is easier to implement than the Red-Black tree because it avoids rotation logic and many different cases, and in this run it also had the lowest mean step count. However, since it depends on random promotions, its performance is less predictable.

In other low-performance systems, I would still choose the AVL tree instead of the Skip List if I wanted more reliable behaviour. The AVL tree is more complex than the Skip List, but it gives consistent results and does not depend on randomness. For this reason, the AVL tree is a good middle ground between the simpler but less predictable Skip List and the more complex Red-Black tree.

Appendix A

Generative AI Usage Appendix

Generative AI was used to assist with the preparation of this assignment. The model used was ChatGPT and was used through the web interface. The assistance was for the following:

- Breaking down the notes and the book into (more) digestible information which can be understood
- Getting a general idea of the different moving parts needed for an implementation of a data structure (especially for the Red-Black Tree). This was checked with online implementation and videos to verify that the output was correct.
- The Formatting of this report; specifically on ideas for layout, and for $\text{\LaTeX}$ help with tables and code snippets